Chapter 5
Mathematical Models for Network Graphs

5.1 Introduction

So far in this book, the emphasis has been almost entirely focused upon methods, to the exclusion of modeling—methods for constructing network graphs, for visualizing network graphs, and for characterizing their observed structure. For the remainder of this book, our focus will shift to the construction and use of models in the analysis of network data, beginning with this chapter, in which we turn to the topic of modeling network graphs.

By a model for a network graph we mean effectively a collection

$$\{\, \mathbb{P}_\theta(G), G \in \mathscr{G} : \theta \in \Theta \,\} \ , \tag{5.1}$$

where $\mathscr{G}$ is a collection (or 'ensemble') of possible graphs, $\mathbb{P}_\theta$ is a probability distribution on $\mathscr{G}$, and θ is a vector of parameters, ranging over possible values in Θ. When convenient, we may often drop the explicit reference to θ and simply write $\mathbb{P}$ for the probability function.

In practice, network graph models are used for a variety of purposes. These include the testing for 'significance' of a pre-defined characteristic(s) in a given network graph, the study of proposed mechanisms for generating certain commonly observed properties in real-world networks (such as broad degree distributions or small-world effects), or the assessment of potential predictive factors of relational ties.

The richness of network graph modeling derives largely from how we choose to specify $\mathbb{P}(\cdot)$, with methods in the literature ranging from the simple to the complex. It is useful for our purposes to distinguish, broadly speaking, between models defined more from (i) a mathematical perspective, versus (ii) a statistical perspective. Those of the former class tend to be simpler in nature and more amendable to mathematical analysis yet, at the same time, do not always necessarily lend themselves well to formal statistical techniques of model fitting and assessment. On the other hand, those of the latter class typically are designed to be fit to data, but their

E.D. Kolaczyk and G. Csárdi, *Statistical Analysis of Network Data with R*, Use R! 65,
DOI 10.1007/978-1-4939-0983-4_5,

mathematical analysis can be challenging in some cases. Nonetheless, both classes of network graph models have their uses for analyzing network graph data.

We will examine certain mathematical models for network graphs in this chapter, and various statistical models, in the following chapter. We consider classical random graph models in Sect. 5.2, and more recent generalizations, in Sect. 5.3. In both cases, these models dictate simply that a graph be drawn uniformly at random from a collection $\mathscr{G}$, but differ in their specification of $\mathscr{G}$. Other classes of network graph models are then presented in Sect. 5.4, where the focus is on models designed to mimic certain observed 'real-world' properties, often through the incorporation of an explicit mechanism(s). Finally, in Sect. 5.5, we look at several examples of ways in which these various mathematical models for network graphs can be used for statistical purposes.

5.2 Classical Random Graph Models

The term *random graph model* typically is used to refer to a model specifying a collection $\mathscr{G}$ and a uniform probability $\mathbb{P}(\cdot)$ over $\mathscr{G}$. Random graph models are arguably the most well-developed class of network graph models, mathematically speaking.

The classical theory of random graph models, as established in a series of seminal papers by Erdős and Rényi [50, 51, 52], rests upon a simple model that places equal probability on all graphs of a given order and size. Specifically, their model specifies a collection $\mathscr{G}_{N_v,N_e}$ of all graphs $G=(V,E)$ with $|V|=N_v$ and $|E|=N_e$, and assigns probability $\mathbb{P}(G)=\binom{N}{N_e}^{-1}$ to each $G\in\mathscr{G}_{N_v,N_e}$, where $N=\binom{N_v}{2}$ is the total number of distinct vertex pairs.

A variant of $\mathscr{G}_{N_v,N_e}$, first suggested by Gilbert [63] at approximately the same time, arguably is seen more often in practice. In this formulation, a collection $\mathscr{G}_{N_v,p}$ is defined to consist of all graphs G of order N_v that may be obtained by assigning an edge independently to each pair of distinct vertices with probability $p\in(0,1)$. Accordingly, this type of model sometimes is referred to as a Bernoulli random graph model. When p is an appropriately defined function of N_v, and $N_e\sim pN_v^2$, these two classes of models are essentially equivalent for large N_v.

The function `erdos.renyi.game` in **igraph** can be used to simulate classical random graphs of either type. Figure 5.1 shows a realization of a classical random graph, based on the choice of $N_v=100$ vertices and a probability of $p=0.02$ of an edge between any pair of vertices. Using a circular layout, we can see that the edges appear to be scattered between vertex pairs in a fairly uniform manner, as expected.

```
#5.1 1 > library(sand)
     2 > set.seed(42)
     3 > g.er <- erdos.renyi.game(100, 0.02)
     4 > plot(g.er, layout=layout.circle, vertex.label=NA)
```

Note that random graphs generated in the manner described above need not be connected.

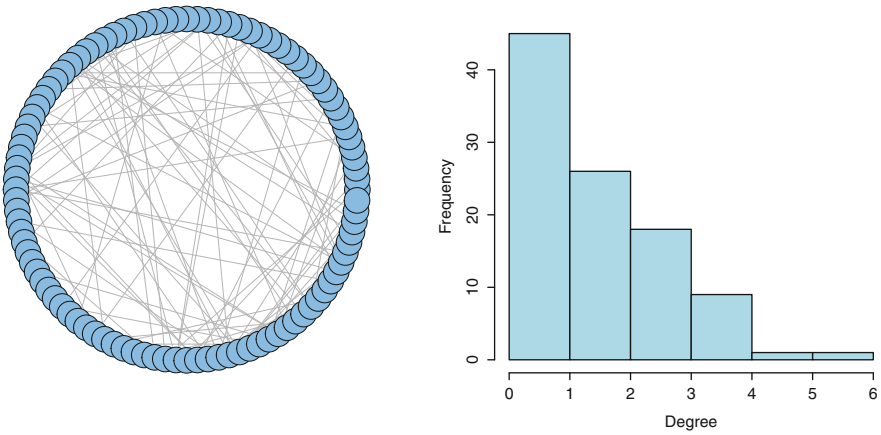

Fig. 5.1 *Left*: A random graph simulated according to an Erdős-Rényi model. *Right*: The corresponding degree distribution

```
#5.2 1 > is.connected(g.er)
     2 [1] FALSE
```

Although this particular realization is not connected, it does nevertheless have a giant component, containing 71 of the 100 vertices. All other components contain between one and four vertices only.

```
#5.3 1 > table(sapply(decompose.graph(g.er), vcount))
     2
     3  1  2  3  4 71
     4 15  2  2  1  1
```

In general, a classical random graph G will with high probability have a giant component if $p = c/N_v$ for some $c > 1$.

Under this same parameterization for p, for $c > 0$, the degree distribution will be well-approximated by a Poisson distribution, with mean c, for large N_v. That this should be true is somewhat easy to see at an intuitive level, since the degree of any given vertex is distributed as a binomial random variable, with parameters $N_v - 1$ and p. The formal proof of this result, however, is nontrivial. The book by Bollobás [16] is a standard reference for this and other such results.

Indeed, in our simulated random graph, the mean degree is quite close to the expected value of $(100 - 1) \times 0.02 = 1.98$.

```
#5.4 1 > mean(degree(g.er))
     2 [1] 1.9
```

Furthermore, in Fig. 5.1 we see that the degree distribution is quite homogeneous.

```
#5.5 1 > hist(degree(g.er), col="lightblue",
     2 +   xlab="Degree", ylab="Frequency", main="")
```

Other properties of classical random graphs include that there are relatively few vertices on shortest paths between vertex pairs

```
#5.6 1 > average.path.length(g.er)
     2 [1] 5.276511
     3 > diameter(g.er)
     4 [1] 14
```

and that there is low clustering.

```
#5.7 1 > transitivity(g.er)
     2 [1] 0.01639344
```

More specifically, under the conditions above, it can be shown that the diameter varies like $O(\log N_v)$, and the clustering coefficient, like N_v^{-1}.

5.3 Generalized Random Graph Models

The formulation of Erdős and Rényi can be generalized in a straightforward manner. Specifically, the basic recipe is to (a) define a collection of graphs $\mathscr{G}$ consisting of all graphs of a fixed order N_v that possess a given characteristic(s), and then (b) assign equal probability to each of the graphs $G \in \mathscr{G}$. In the Erdős-Rényi model, for example, the common characteristic is simply that the size of the graphs G be equal to some fixed N_e.

Beyond Erdős-Rényi, the most commonly chosen characteristic is that of a fixed degree sequence. That is, $\mathscr{G}$ is defined to be the collection of all graphs G with a pre-specified degree sequence, which we will write here as $\{d_{(1)}, \ldots, d_{(N_v)}\}$, in ordered form.

The **igraph** function `degree.sequence.game` can be used to uniformly sample random graphs with fixed degree sequence. Suppose, for example, that we are interested in graphs of $N_v = 8$ vertices, half of which have degree $d = 2$, and the other half, degree $d = 3$. Two examples of such graphs, drawn uniformly from the collection of all such graphs, are shown in Fig. 5.2.

```
#5.8 1 > degs <- c(2,2,2,2,3,3,3,3)
     2 > g1 <- degree.sequence.game(degs, method="vl")
     3 > g2 <- degree.sequence.game(degs, method="vl")
     4 > plot(g1, vertex.label=NA)
     5 > plot(g2, vertex.label=NA)
```

Note that these two graphs do indeed differ, in that they are not isomorphic.[1]

```
#5.9 1 > graph.isomorphic(g1, g2)
     2 [1] FALSE
```

For a fixed number of vertices N_v, the collection of random graphs with fixed degree sequence all have the same number of edges N_e, due to the relation $\bar{d} = 2N_e/N_v$, where $\bar{d}$ is the mean degree of the sequence $(d_{(1)}, \ldots, d_{(N_v)})$.

```
#5.10 1 > c(ecount(g1), ecount(g2))
      2 [1] 10 10
```

[1] Recall that two graphs are said to be *isomorphic* if they differ only up to relabellings of their vertices and edges that leave the structure unchanged.

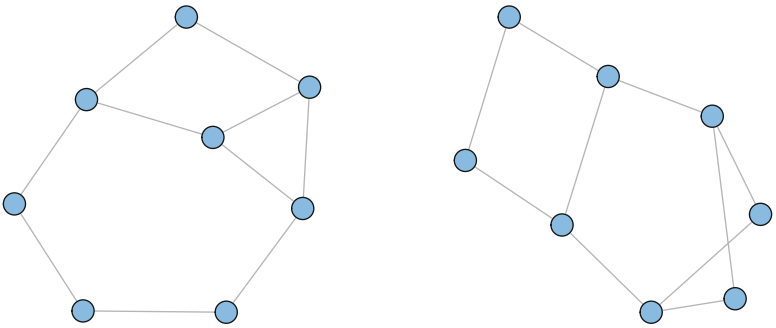

Fig. 5.2 Two graphs drawn uniformly from the collection of all graphs G with $N_v = 8$ vertices and degree sequence $(2,2,2,2,3,3,3,3)$

Therefore, this collection is strictly contained within the collection of random graphs $\mathscr{G}_{N_v,N_e}$, with the corresponding number N_v, N_e of vertices and edges. So the addition of an assumed form for the degree sequence is in this case equivalent to specifying our model through a conditional distribution on the original collection $\mathscr{G}_{N_v,N_e}$.

On the other hand, it is important to keep in mind that all other characteristics are free to vary to the extent allowed by the chosen degree sequence. For example, we can generate a graph with the same degree sequence as our network of protein–protein interactions in yeast.

```
#5.11 1 > data(yeast)
      2 > degs <- degree(yeast)
      3 > fake.yeast <- degree.sequence.game(degs,
      4 +   method=c("vl"))
      5 > all(degree(yeast) == degree(fake.yeast))
      6 [1] TRUE
```

But the original network has twice the diameter of the simulated version

```
#5.12 1 > diameter(yeast)
      2 [1] 15
      3 > diameter(fake.yeast)
      4 [1] 7
```

and virtually all of the substantial amount of clustering originally present is now gone.

```
#5.13 1 > transitivity(yeast)
      2 [1] 0.4686178
      3 > transitivity(fake.yeast)
      4 [1] 0.03968903
```

In principle, it is easy to further constrain the definition of the class $\mathscr{G}$, so that additional characteristics beyond the degree sequence are fixed. Markov chain Monte Carlo (MCMC) methods are popular for generating generalized random graphs G from such collections, where the states visited by the Markov chain are the distinct graphs G themselves. Snijders [133], Roberts [124], and more recently, McDonald,

Smith, and Forster [108], for example, offer MCMC algorithms for a number of different cases, such as where the graphs G are directed and constrained to have both a fixed pair of in- and out-degree sequences and a fixed number of mutual arcs. However, the design of such algorithms in general is nontrivial, and development of the corresponding theory, in turn, which would allow users to verify the assumptions underlying Markov chain convergence, has tended to lag somewhat behind the pace of algorithm development.

5.4 Network Graph Models Based on Mechanisms

Arguably one of the more important innovations in modern network graph modeling is a movement from traditional random graph models, like those in the previous two sections, to models explicitly designed to mimic certain observed 'real-world' properties, often through the incorporation of a simple mechanism(s). Here in this section we look at two canonical classes of such network graph models.

5.4.1 Small-World Models

Work on modeling of this type received a good deal of its impetus from the seminal paper of Watts and Strogatz [146] and the 'small-world' network model introduced therein. These authors were intrigued by the fact that many networks in the real world display high levels of clustering, but small distances between most nodes. Such behavior cannot be reproduced, for example, by classical random graphs, since, recall that, while the diameter scales like $O(\log N_v)$, indicating small distances between nodes, the clustering coefficient behaves like N_v^{-1}, which suggests very little clustering.

In order to create a network graph with both of these properties, Watts and Strogatz suggested instead beginning with a graph with lattice structure, and then randomly 'rewiring' a small percentage of the edges. More specifically, in this model we begin with a set of N_v vertices, arranged in a periodic fashion, and join each vertex to r of its neighbors to each side. Then, for each edge, independently and with probability p, one end of that edge will be moved to be incident to another vertex, where that new vertex is chosen uniformly, but with attention to avoid the construction of loops and multi-edges.

An example of a small-world network graph of this sort can be generated in **igraph** using the function `watts.strogatz.game`.

```
#5.14 1 > g.ws <- watts.strogatz.game(1, 25, 5, 0.05)
      2 > plot(g.ws, layout=layout.circle, vertex.label=NA)
```

The resulting graph, with $N_v = 25$ vertices, neighborhoods of size $r = 5$, and rewiring probability $p = 0.05$, is shown in Fig. 5.3.

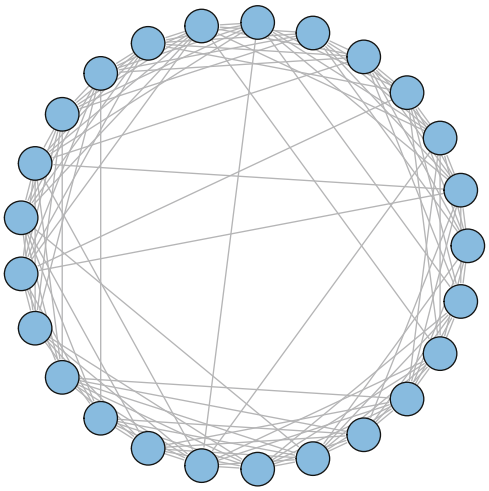

Fig. 5.3 Example of a Watts-Strogatz 'small-world' network graph

For the lattice alone, which we generate by setting $p = 0$, there is a substantial amount of clustering.

```
#5.15 1 > g.lat100 <- watts.strogatz.game(1, 100, 5, 0)
      2 > transitivity(g.lat100)
      3 [1] 0.6666667
```

But the distance between vertices is non-trivial.

```
#5.16 1 > diameter(g.lat100)
      2 [1] 10
      3 > average.path.length(g.lat100)
      4 [1] 5.454545
```

The effect of rewiring a relatively small number of edges in a random fashion is to noticeably reduce the distance between vertices, while still maintaining a similarly high level of clustering.

```
#5.17 1 > g.ws100 <- watts.strogatz.game(1, 100, 5, 0.05)
      2 > diameter(g.ws100)
      3 [1] 4
      4 > average.path.length(g.ws100)
      5 [1] 2.669091
      6 > transitivity(g.ws100)
      7 [1] 0.4864154
```

This effect may be achieved even with very small p. To illustrate, we simulate according to a particular Watts-Strogatz small-world network model, with $N_v = 1,000$ and $r = 10$, and re-wiring probability p, as p varies over a broad range.

```
#5.18 1 > steps <- seq(-4, -0.5, 0.1)
      2 > len <- length(steps)
      3 > cl <- numeric(len)
      4 > apl <- numeric(len)
      5 > ntrials <- 100
      6 > for (i in (1:len)) {
      7 +   cltemp <- numeric(ntrials)
      8 +   apltemp <- numeric(ntrials)
      9 +   for (j in (1:ntrials)) {
     10 +     g <- watts.strogatz.game(1, 1000, 10, 10^steps[i])
     11 +     cltemp[j] <- transitivity(g)
     12 +     apltemp[j] <- average.path.length(g)
     13 +   }
     14 +   cl[i] <- mean(cltemp)
     15 +   apl[i] <- mean(apltemp)
     16 + }
```

The results shown in Fig. 5.4, where approximate expected values for normalized versions of average path length and clustering coefficient are plotted, indicate that over a substantial range of p the network exhibits small average distance while maintaining a high level of clustering.

```
#5.19 1 > plot(steps, cl/max(cl), ylim=c(0, 1), lwd=3, type="l",
      2 +   col="blue", xlab=expression(log[10](p)),
      3 +   ylab="Clustering and Average Path Length")
      4 > lines(steps, apl/max(apl), lwd=3, col="red")
```

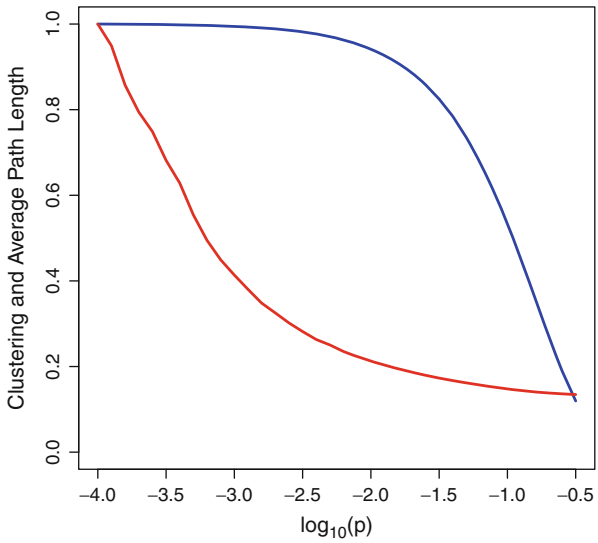

Fig. 5.4 Plot of the clustering coefficient $\mathrm{cl}_T(G)$ (*blue*) and average path length (*red*), as a function of the rewiring probability p for a Watts-Strogatz small-world model. Results are averages based on 100 simulation trials

5.4.2 *Preferential Attachment Models*

Many networks grow or otherwise evolve in time. The World Wide Web and scientific citation networks are two obvious examples. Similarly, many biological networks may be viewed as evolving as well, over appropriately defined time scales. Much energy has been invested in the development of models that mimic network growth.

In this arena, typically a simple mechanism(s) is specified for how the network changes at any given point in time, based on concepts like vertex preference, fitness, copying, age, and the like. A celebrated example of such a mechanism is that of *preferential attachment*, designed to embody the principle that 'the rich get richer.' A driving motivation behind the introduction of this particular mechanism was a desire to reproduce the types of broad degree distributions observed in many large, real-world networks. Although there were a number of precursors with similar ideas, it is the work of Barabási and Albert [8] that launched the present-day fascination with models of this type.[2]

Barabási and Albert were motivated by the growth of the World Wide Web, noting that often web pages to which many other pages point will tend to accumulate increasingly greater numbers of links as time goes on. The Barabási-Albert (BA) model for undirected graphs is formulated as follows. Start with an initial graph $G^{(0)}$ of $N_v^{(0)}$ vertices and $N_e^{(0)}$ edges. Then, at stage $t = 1, 2, \ldots$, the current graph $G^{(t-1)}$ is modified to create a new graph $G^{(t)}$ by adding a new vertex of degree $m \geq 1$, where the m new edges are attached to m different vertices in $G^{(t-1)}$, and the probability that the new vertex will be connected to a given vertex v is given by

$$\frac{d_v}{\sum_{v' \in V} d_{v'}} \quad . \tag{5.2}$$

That is, at each stage, m existing vertices are connected to a new vertex in a manner preferential to those with higher degrees. After t iterations, the resulting graph $G^{(t)}$ will have $N_v^{(t)} = N_v^{(0)} + t$ vertices and $N_e^{(t)} = N_e^{(0)} + tm$ edges. And, because of the tendency towards preferential attachment, intuitively we would expect that a number of vertices of comparatively high degree should gradually emerge as t increases.

Using the **igraph** function `barabasi.game`, we can simulate a BA random graph of, for example, $N_v = 100$ vertices, with $m = 1$ new edges added for each new vertex.

```
#5.20 1 > set.seed(42)
      2 > g.ba <- barabasi.game(100, directed=FALSE)
```

A visualization of this graph is shown in Fig. 5.5.

```
#5.21 1 > plot(g.ba, layout=layout.circle, vertex.label=NA)
```

[2] For an extensive history of power-law models, including their use in network modeling, see Mitzenmacher [112].

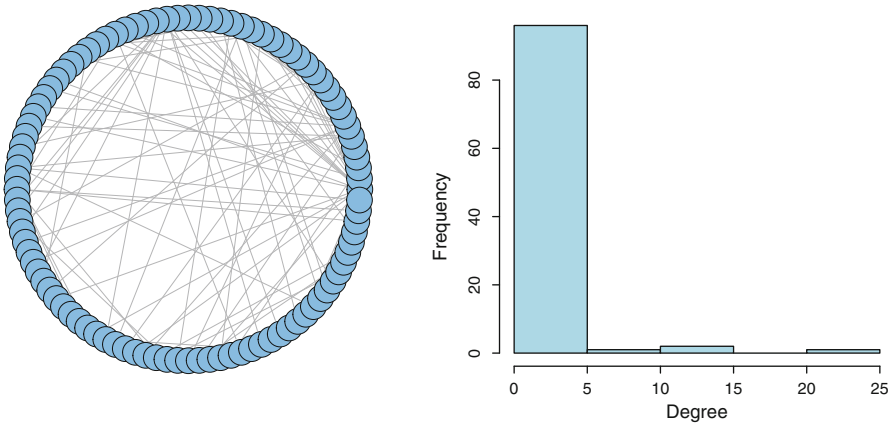

Fig. 5.5 *Left*: A random graph simulated according to a Barabási-Albert model. *Right*: The corresponding degree distribution

Note that the edges are spread among vertex pairs in a decidedly less uniform manner than in the classical random graph we saw in Fig. 5.1. And, in fact, there appear to be vertices of especially high degree—so-called 'hub' vertices.

Examination of the degree distribution (also shown in Fig. 5.5)

```
#5.22 1 > hist(degree(g.ba), col="lightblue",
      2 +   xlab="Degree", ylab="Frequency", main="")
```

confirms this suspicion, and indicates, moreover, that the overall distribution is quite heterogeneous. Actually, the vast majority of vertices have degree no more than two in this graph, while, on the other hand, one vertex has a degree of 11.

```
#5.23 1 > summary(degree(g.ba))
      2    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
      3    1.00    1.00    1.00    1.98    2.00   11.00
```

The most celebrated property of such preferential attachment models is that, in the limit as t tends to infinity, the graphs $G^{(t)}$ have degree distributions that tend to a power-law form $d^{-\alpha}$, with $\alpha = 3$. See [8, 48, 93, 19]. This behavior is in noted contrast to the case of classical random graphs.

On the other hand, network graphs generated according to the BA model will share with their classical counterparts the tendency towards relatively few vertices on shortest paths between vertex pairs

```
#5.24 1 > average.path.length(g.ba)
      2 [1] 4.923434
      3 > diameter(g.ba)
      4 [1] 9
```

and low clustering.

```
#5.25 1 > transitivity(g.ba)
      2 [1] 0
```

See [18, 17] for details.

5.5 Assessing Significance of Network Graph Characteristics

As we remarked at the start of this chapter, the network graph models we have described above generally are too simple for serious statistical modeling with observed networks. Nonetheless, from a statistical hypothesis testing perspective, they still have a useful role to play in network analysis. Specifically, the network models introduced in this chapter are frequently used in the assessment of significance of network graph characteristics.

Suppose that we have a graph derived from observations of some sort, which we will denote as G^{obs} here, and that we are interested in some structural characteristic, say $\eta(\cdot)$. In many contexts it is natural to ask whether $\eta(G^{obs})$ is 'significant,' in the sense of being somehow unusual or unexpected. Network models of the type we have considered so far are used in establishing a well-defined frame of reference. That is, for a given collection of graphs $\mathscr{G}$, the value $\eta(G^{obs})$ is compared to the collection of values $\{\eta(G) : G \in \mathscr{G}\}$. If $\eta(G^{obs})$ is judged to be extreme with respect to this collection, then that is taken as evidence that G^{obs} is unusual in having this value.

When random graph models are used, it is straightforward to create a formal reference distribution which, under the accompanying assumption of uniform probability of elements in $\mathscr{G}$, takes the form

$$\mathbb{P}_{\eta,\mathscr{G}}(t) = \frac{\#\{G \in \mathscr{G} : \eta(G) \leq t\}}{|\mathscr{G}|} \quad . \tag{5.3}$$

If $\eta(G^{obs})$ is found to be sufficiently unlikely under this distribution, this is taken as evidence against the hypothesis that G^{obs} is a uniform draw from $\mathscr{G}$.

This principle lies at the heart of methods aimed at the detection of *network motifs*, defined by Alon and colleagues [87, 111] to be small subgraphs occurring far more frequently in a given network than in comparable random graphs. We refer the reader to these articles for details. Also see [91, Chap. 6.2].

Here we demonstrate similar use of this principle through two somewhat simpler examples.

5.5.1 Assessing the Number of Communities in a Network

Recall from Chap. 4.4.1 that our use of hierarchical clustering, through the function `fastgreedy.community`, resulted in the discovery of three communities in the

karate network. We might ask ourselves whether this is in some sense unexpected or unusual. In defining our frame of reference we will use two choices of $\mathscr{G}$: (i) graphs of the same order $N_v = 34$ and size $N_e = 78$ as the karate network, and (ii) graphs that obey the further restriction that they have the same degree distribution as the original.

Monte Carlo methods, using some of the same R functions encountered above, allow us to quickly generate approximations to the corresponding reference distributions. In order to do so, we need the order, size, and degree sequence of the original karate network.

```
#5.26 1 > data(karate)
      2 > nv <- vcount(karate)
      3 > ne <- ecount(karate)
      4 > degs <- degree(karate)
```

Over 1000 trials,

```
#5.27 1 > ntrials <- 1000
```

we then generate classical random graphs of this same order and size and, for each one, we use the same community detection algorithm to determine the number of communities.

```
#5.28 1 > num.comm.rg <- numeric(ntrials)
      2 > for(i in (1:ntrials)){
      3 +   g.rg <- erdos.renyi.game(nv, ne, type="gnm")
      4 +   c.rg <- fastgreedy.community(g.rg)
      5 +   num.comm.rg[i] <- length(c.rg)
      6 + }
```

Similarly, we do the same using generalized random graphs constrained to have the required degree sequence.

```
#5.29 1 > num.comm.grg <- numeric(ntrials)
      2 > for(i in (1:ntrials)){
      3 +   g.grg <- degree.sequence.game(degs, method="vl")
      4 +   c.grg <- fastgreedy.community(g.grg)
      5 +   num.comm.grg[i] <- length(c.grg)
      6 + }
```

The results may be summarized and compared using side by side bar plots.

```
#5.30 1 > rslts <- c(num.comm.rg,num.comm.grg)
      2 > indx <- c(rep(0, ntrials), rep(1, ntrials))
      3 > counts <- table(indx, rslts)/ntrials
      4 > barplot(counts, beside=TRUE, col=c("blue", "red"),
      5 +   xlab="Number of Communities",
      6 +   ylab="Relative Frequency",
      7 +   legend=c("Fixed Size", "Fixed Degree Sequence"))
```

See Fig. 5.6. Clearly the actual number of communities detected in the original karate network (i.e., three) would be considered unusual from the perspective of random graphs of both fixed size and fixed degree sequence. Accordingly, we may

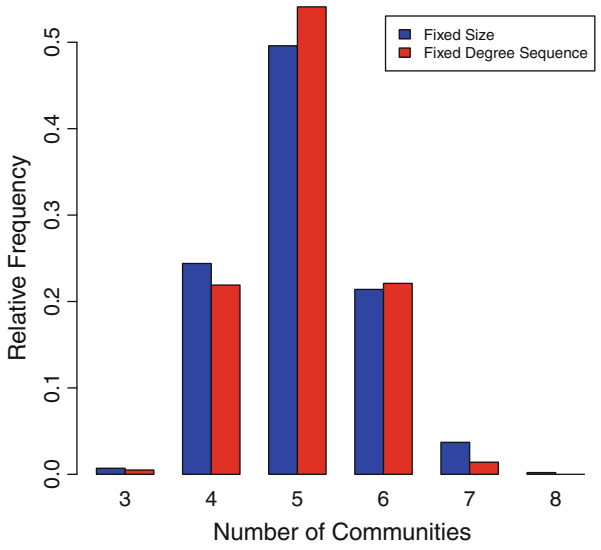

Fig. 5.6 Distribution of number of communities detected for random graphs of the same size (*blue*) and degree sequence (*red*) as the karate network

conclude that there is likely an additional mechanism(s) at work in the actual karate club, one that goes beyond simply the density and the distribution of social interactions in this network.

5.5.2 Assessing Small World Properties

The notion of small-world networks has been particularly popular in the field of neuroscience, where numerous authors have presented evidence for arguments that small-world behavior can be found in various network-based representations of the brain. See [10] for a review. On the other hand, such claims are not completely uncontested, with recent work arguing that—in certain contexts—some or all of the observed behavior may be attributed to aspects of the sampling underlying the data used to construct such networks (e.g., [14, 61]).

A typical approach to assessing small-world behavior in this area is to compare the observed clustering coefficient and average (shortest) path length in an observed network to what might be observed in an appropriately calibrated classical random graph. Recalling our discussion of small world networks in Sect. 5.4.1, we should expect under such a comparison—if indeed an observed network exhibits small-world behavior—that the observed clustering coefficient exceed that of a random graph, while the average path length remain roughly the same.

The data set `macaque` in **igraph** contains a network of established functional connections between brain areas understood to be involved with the tactile function of the visual cortex in macaque monkeys [115].

```
#5.31 1 > library(igraphdata)
      2 > data(macaque)
      3 > summary(macaque)
      4 IGRAPH DN-- 45 463 --
      5 attr: Citation (g/c), Author (g/c), shape (v/c),
      6   name (v/c)
```

It is a directed network of 45 vertices and 463 links.

In order to assess clustering in this network, we use an extension to directed graphs of the clustering coefficient defined in Chap. 4.3.2, due to [53]. This quantity is defined as the average, over all vertices v, of the vertex-specific clustering coefficients

$$\mathsf{cl}(v) = \frac{\left(\mathbf{A}+\mathbf{A}^T\right)^3_{vv}}{2\left[d_v^{tot}\left(d_v^{tot}-1\right)-2\left(\mathbf{A}^2\right)_{vv}\right]} \quad , \tag{5.4}$$

where $\mathbf{A}$ is the adjacency matrix and d_v^{tot} is the total degree (i.e., in-degree plus out-degree) of vertex v. The calculation of this quantity is accomplished through the following function.

```
#5.32 1 > clust.coef.dir <- function(graph) {
      2 +   A <- as.matrix(get.adjacency(graph))
      3 +   S <- A + t(A)
      4 +   deg <- degree(graph, mode=c("total"))
      5 +   num <- diag(S %*% S %*% S)
      6 +   denom <- diag(A %*% A)
      7 +   denom <- 2 * (deg * (deg - 1) - 2 * denom)
      8 +   cl <- mean(num / denom)
      9 +   return(cl)
     10 + }
```

Similarly, path lengths are defined in this network only for directed paths.

The steps required to simulate draws of (directed) classical random graphs, and to assess the clustering and average path length for each, are analogous to those employed in our example with the karate network just above.

```
#5.33 1 > ntrials <- 1000
      2 > nv <- vcount(macaque)
      3 > ne <- ecount(macaque)
      4 > cl.rg <- numeric(ntrials)
      5 > apl.rg <- numeric(ntrials)
      6 > for (i in (1:ntrials)) {
      7 +   g.rg <- erdos.renyi.game(nv, ne, type="gnm",
      8 +     directed=TRUE)
      9 +   cl.rg[i] <- clust.coef.dir(g.rg)
     10 +   apl.rg[i] <- average.path.length(g.rg)
     11 + }
```

Summarizing the resulting distributions of clustering coefficient

```
#5.34 1 > summary(cl.rg)
      2    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
      3  0.2159  0.2302  0.2340  0.2340  0.2377  0.2548
```

and average path length

```
#5.35 1 > summary(apl.rg)
      2    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
      3   1.810   1.827   1.833   1.833   1.838   1.858
```

and comparing against these distributions the values for the macaque network

```
#5.36 1 > clust.coef.dir(macaque)
      2 [1] 0.5501073
      3 > average.path.length(macaque)
      4 [1] 2.148485
```

we find that our observed values fall far outside the range typical of these random graphs.

Therefore, we see that on the one hand there is substantially more clustering in our network than expected from a random network. On the other hand, however, the shortest paths between vertex pairs are, on average, also noticeably longer. Hence, the evidence for small-world behavior in this network is not clear, with the results suggesting that the network behaves more like a lattice than a classical random graph.

5.6 Additional Reading

There are a number of books devoted entirely to subsets of the topics in network graph modeling discussed in this chapter. For example, the volume by Bollobás [16] is the standard reference for classical random graph theory, while the book of Chung and Lu [29] contains a unified exposition of numerous formal results on generalized random graph, network growth, and small-world models. Similarly, many books on network analysis include such material as well, to varying extents. See, for example, the book by Newman [118, Sect. IV].